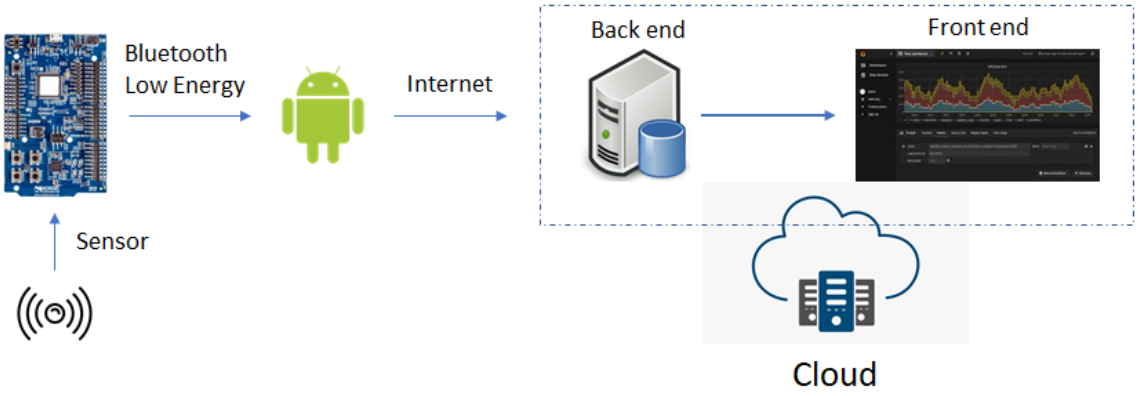
Internet of Things Labs

During these lab sessions, you will work in groups of three on an IoT project. You will send data from an electronic device to the cloud via an intermediate device, Edge.

During these sessions, you will be able to use the knowledge you have acquired in other subjects you have studied in previous years, such as project management, programming, AI model training, etc.

We will assume that your group is a company, and I will hire you to carry out an IoT project. This project will have minimum requirements, which will be evaluated at the end of the course (Below is an outline and summary of what you will implement during the practice sessions). As a group, you will have to manage the implementation of the project.



You can make a small-scale implementation of your design project. As it may not be possible to implement everything you propose in the theoretical design document, you can make a simulation of the system, generate data and transmitting it from the Device to the Cloud.

Device (Sensor and PCB)

Sensors are devices that capture events in our environment. It can be a button, an accelerometer or a light, sound, temperature, or humidity sensor.

They are connected to a microcontroller that is responsible of receiving sensor data and processing them. In our case we will use Bluetooth Low Energy (BLE) to extract the data captured by the sensor.

During the course, we lend each group a Nordic nRF52840-DK (Quick Start — nRF Connect SDK 3.4.99 documentation)

Edge (Android mobile app or Raspberry pi or Laptop)

An application will connect to the PCB via BLE and receive the data. These can be processed on the device or sent to a server over the Internet.

It is proposed to use android since 70% of mobile phones use this operating system. In case you have an iPhone and a Mac laptop, you can develop an application for this system.

Another option is to use an embedded platform instead of a mobile like Edge. For example, you can use a Raspberry pi and develop BLE communication on this platform.

Cloud

In a system where data is captured, the most important thing is to be able to store and display it. That is why we need a database with a back end and a system that represents this data, for example a graph on the web, front end.

With this data we can perform computational tasks such as prediction or detection.

(As a Cloud you can use a commercial platform or create a server on your own computer to simulate it).

Tasks:

- Program de PCB and send data via BLE
- Receive data from BLE in an Android app or Edge device
- Send data from Edge to Cloud
- Store data on the cloud server
- Create some ML model with the data at Edge or/and Cloud level.
- Show data to final users

Embedded system Example (Nordic nRF52)

For PCB programming, you can use:

- Nordic provide with different tool (nRF Connect) and a plugin for VSCode to program devices
- PlatformIO (Your Gateway to Embedded Software Development Excellence · PlatformIO). It is a framework that allows you to transparently program different embedded processors without needing to know the specifics of each manufacturer. Allows you to use Arduino, ZephyrRTOS, and Mbed.
  - ***Arduino (deprecated – Errors in BLE library since 2024)***
  - ***Zephyr***

Step by step to run bluetooth through Zephyr (v2.7) on the Nrf52 board:

- Download the repository https://github.com/platformio/platform-nordicnrf52/tree/master
- Unzip and go to the folder “example/zephyr-ble-beacon” and copy it to your working directory.
- In visual Studio Code “Open project” and select the above folder.
- In the “platform.ini” file delete the “particle_xenon” board block.
- Compile and upload.
- In your mobile phone, using the NRFConnect app you should see a beacon “Test beacon”.

Example Android BLE (Java):
(https://developer.android.com/guide/topics/connectivity/bluetooth/ble-overview
Java and Kotlin documentation and example)

Android applications can be used as Edge devices in the IoT architecture. Students may implement their mobile application using either **Kotlin** (recommended) or **Java**.

1. Permissions Configuration (AndroidManifest.xml)

To prevent runtime permission issues on modern Android devices (Android 12 / API level 31+), ensure your project declares the updated BLE permissions in the manifest:

<uses-permission android:name="android.permission.BLUETOOTH_SCAN" android:usesPermissionFlags="neverForLocation" />

<uses-permission android:name="android.permission.BLUETOOTH_CONNECT" />

<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />

(For devices running Android 11 or lower, standard legacy Bluetooth and location permissions apply).

2. Technical References & Official Documentation

Students are expected to consult the official Android Developers documentation to design their BLE stack (scanning, connection, and data acquisition):

- **BLE Scanning**: Consult BluetoothLeScanner and ScanCallback APIs to locate your PCB device using your assigned Service UUID.
- **GATT Connection & Service Discovery**: Use BluetoothGatt and BluetoothGattCallback to establish the connection and discover available services and characteristics.
- **Notifications**: Enable characteristic notifications locally and write to the Client Characteristic Configuration Descriptor (CCCD: 00002902-0000-1000-8000-00805f9b34fb) to receive continuous sensor updates from the PCB.

Example Cloud platform:

- Amazon
- Google cloud
- Azure
- Oracle (free tier 0 VM)
- Simulate cloud behavior on your computer installing Back-end/Web/DB server.

Sample tools that you can use to create the system. (You can use whatever tools you want; the list is just a guide)

PCB:

- Platformio.org (Arduino, ZephyrRTOS, Mbed) (VSCode)
- Nordic SDK (VSCode)

Android:

- Android Studio

- Xamarin
- Flutter
- Ionic

Back-end:

- Firebase
- MQTT
- Influx
- AWS Amplify

Front-end:

- HTML5/CSS/JS framework
- Mobile

Evaluation

Final score = 1st report (20%) + 2nd report (20%) + project (60%)

Groups B = 8 to 14
Groups A = 1 to 7

- First Report, task done on PCB and Android regarding BLE communication. Submission through the Virtual Campus:
  - **30th of October**
- Evaluation of the projects in the LAB (Any change on the date will be published in Virtual Campus)
  - **7th of January**
- **10th of January** - 2nd report with back-end and Front-end + zip with code of the project or github repository. Submission through the Virtual Campus.

Project Deliverables & Report Guidelines

Both project reports must follow a professional technical format. As your group operates as a company delivering an IoT product, your documentation must clearly detail your design choices, custom protocol specifications, and validation steps.

Required Report Structure

1. **Executive Summary & Project Overview**
   - High-level description of the company, the IoT product, and the target application/use case.
   - Summary of the project goals and system capabilities.
2. **System Architecture & Diagram**
   - **Complete Block Diagram**: A high-level visual representation showing all three layers: **Device (Sensors & PCB)**, **Edge (Mobile/Gateway)**, and Cloud (Backend/Frontend/ML).
   - Identification of interfaces, data flows, and communication channels between layers.

3. **Layer Implementation Details**

    - **Device Layer**: Microcontroller board used, sensor hardware, firmware frameworks (e.g., PlatformIO with Zephyr or Arduino), hardware interrupts, and power management.
    - **Edge Layer**: Platform choice (Android app in Kotlin/Java, Raspberry Pi, laptop), OS version, and BLE scanning/connection management.
    - **Cloud Layer**: Backend technology, database strategy (Time-Series, Relational, NoSQL), web UI framework, and Machine Learning model design/purpose.
    - **Libraries & Frameworks**: Explicit list of third-party dependencies, APIs, and libraries used at each stage.

4. **Data Structures & Protocol Payload Specifications**

    - Since each group defines its own communication strategy, you must explicitly document your payload structures:
        - **BLE Data Frame**: Precise byte breakdown of the data packet sent over BLE (e.g., header, sensor payload fields, timestamp, CRC/checksum).
        - **Edge-to-Cloud Data Format**: Protocol used (e.g., MQTT, HTTP REST, WebSockets) and JSON/payload schema used for data ingestion.

5. **Testing, Validation & Results**

    - **Per-Layer Testing**: Specific unit and functional tests executed to verify each component (PCB sensor reading accuracy, BLE connection stability, API endpoint response times).
    - **End-to-End Testing**: Validation of complete data flows from physical measurement at the PCB to visualization on the web dashboard.
    - **Test Evidence**: Include graphs, logs, or screenshots demonstrating operational stability.

Team Assessment & Course Feedback

To encourage professional responsability and continuously improve the course, the final section of the second report must include a **Team & Self-Assessment** along **Course Feedback**.

1. Teamwork & Individual Contribution

Every group member must contribute meaningfully to the project. In this subsection, clearly detail:

- **Task Distribution Matrix**: Provide a breakdown listing the specific responsibilities, modules, and code components led by each member (e.g., Firmware/PCB, Android Edge app, Backend API, Frontend UI, ML model).
- **Self-Assessment**: Evaluate your own technical growth, problem-solving approach, and personal commitment to the team's objectives.

- **Peer Evaluation & Collaboration**: Describe how your team organized its workflow (e.g., version control, task tracking, meeting frequency), how decisions were made, and how any technical or interpersonal challenges were resolved.

2. Course & Subject Feedback

Your constructive feedback is essential for refining future editions of this laboratory course. Please provide honest reflections on:

- **Strengths**: What aspects of the project, lab exercises, hardware tools, or teaching methodology were most engaging and beneficial for your learning?
- **Areas for Improvement**: What specific elements would you modify or expand? (e.g., clarity of instructions, suggested frameworks, hardware resources, schedule, or evaluation criteria).